

2

Processos de software

OBJETIVOS

O objetivo deste capítulo é introduzir o conceito de processo de software — um conjunto coerente de atividades para produção de software. Ao ler este capítulo, você:

- ▶ compreenderá os conceitos e os modelos de processo de software;
- ▶ será apresentado a três modelos genéricos de processo de software e às situações nas quais eles podem ser utilizados;
- ▶ conhecerá as atividades de processo fundamentais da engenharia de requisitos, do desenvolvimento, dos testes e da evolução de software;
- ▶ compreenderá por que os processos devem ser organizados para lidar com as mudanças nos requisitos e no projeto de software;
- ▶ compreenderá o conceito de melhoria do processo de software e os fatores que afetam a qualidade do processo.

CONTEÚDO

- 2.1 Modelos de processo de software
- 2.2 Atividades do processo
- 2.3 Lidando com mudanças
- 2.4 Melhoria de processo

Um processo de software é um conjunto de atividades relacionadas que levam à produção de um sistema de software. Conforme discutimos no Capítulo 1, existem muitos tipos diferentes de sistemas de software e não há um método universal de engenharia de software que seja aplicável a todos eles. Consequentemente, não existem processos de software universalmente aplicáveis. O processo utilizado nas diferentes empresas depende do tipo de software que está sendo desenvolvido, dos requisitos do cliente e das habilidades das pessoas que o desenvolvem.

No entanto, embora existam muitos processos de software diferentes, todos eles devem incluir, de alguma forma, as quatro atividades fundamentais da engenharia de software que introduzi no Capítulo 1:

1. *Especificação.* A funcionalidade do software e as restrições sobre sua operação devem ser definidas.
2. *Desenvolvimento.* O software deve ser produzido para atender à especificação.
3. *Validação.* O software deve ser validado para garantir que atenda ao que o cliente deseja.
4. *Evolução.* O software deve evoluir para atender às mudanças nas necessidades dos clientes.

Essas atividades são complexas por si só e incluem subatividades como validação dos requisitos, projeto da arquitetura e teste de unidade. Os processos incluem, ainda, atividades como gerenciamento de configuração do software e planejamento de projeto, que também apoiam as atividades de produção.

Quando descrevemos e discutimos os processos, normalmente falamos sobre as atividades nesses processos — especificar um modelo de dados e projetar uma interface com o usuário, por exemplo — e sobre a sequência correta dessas atividades. Todos nós podemos identificar o que as pessoas fazem para desenvolver um software, mas, quando se trata de processos de software, é importante descrever também quem está envolvido, o que está sendo produzido e quais condições influenciam a sequência dessas atividades:

1. Uma atividade de processo resulta em produtos ou em entregas. Por exemplo, o resultado da atividade de projeto da arquitetura pode ser um modelo da arquitetura do software.
2. Os papéis refletem as responsabilidades das pessoas envolvidas no processo. Entre os exemplos de papéis, temos os do gerente de projeto, do gerente de configuração e do programador.
3. Há condições que devem ser mantidas antes ou depois de uma atividade do processo ter sido aprovada ou um produto ter sido produzido. Antes de começar o projeto da arquitetura, por exemplo, uma pré-condição poderia ser a de que o consumidor tenha aprovado todos os requisitos; depois que essa atividade for concluída, uma pós-condição poderia ser a de que os modelos em UML descrevendo a arquitetura fossem revisados.

Os processos de software são complexos e, como processos intelectuais e criativos, dependem da tomada de decisão e do julgamento das pessoas. Uma vez que não existe um processo universal que valha para todos os tipos de software, a maioria das empresas produtoras de software concebeu seus próprios processos de desenvolvimento. Estes evoluíram e passaram a aproveitar a capacidade dos desenvolvedores de software em uma organização e as características dos sistemas que estão sendo desenvolvidos. No caso dos sistemas críticos em segurança, é necessário um processo de desenvolvimento muito estruturado e que registros detalhados sejam mantidos. Nos sistemas de negócio, com requisitos que mudam rapidamente, talvez seja melhor adotar um processo mais flexível e ágil.

Conforme discutimos no Capítulo 1, o desenvolvimento de software profissional é uma atividade gerenciada, logo, o planejamento é parte inerente a todos os processos. Os processos dirigidos por planos são aqueles em que todas as atividades são planejadas antecipadamente e progridem em relação ao que foi planejado. Nos processos ágeis, que discuto no Capítulo 3, o planejamento é incremental e contínuo à medida que o software é desenvolvido. Portanto, nesses

casos, é mais fácil mudar o processo para refletir a mudança dos requisitos do cliente ou do produto. Conforme explicam Boehm e Turner (2004), cada abordagem é adequada para diferentes tipos de software. Geralmente, nos sistemas grandes, é preciso encontrar um equilíbrio entre os processos dirigidos por planos e os processos ágeis.

Embora não haja um processo de software universal, há espaço para a melhoria dos processos em muitas organizações. Os processos podem incluir técnicas ultrapassadas ou não tirar proveito da prática mais recomendada na engenharia de software industrial. Na realidade, durante seus processos de desenvolvimento, muitas organizações ainda não se valem dos métodos de engenharia de software. Elas podem melhorar seus processos introduzindo técnicas como a modelagem com UML e o desenvolvimento dirigido por testes. Discutirei brevemente a melhoria dos processos de software mais adiante neste capítulo, e mais detalhadamente no Capítulo 26, disponível no site de apoio (em inglês).

2.1 MODELOS DE PROCESSO DE SOFTWARE

Conforme expliquei no Capítulo 1, um modelo de processo de software — às vezes chamado de ciclo de vida do desenvolvimento de software (ou modelo SDLC, do inglês *Software Development Life Cycle*) — é uma representação simplificada de um processo de software. Cada modelo representa um processo a partir de uma perspectiva particular e, desse modo, fornece apenas informações parciais sobre esse processo. Por exemplo, um modelo de atividades do processo mostra as atividades e sua sequência, mas não os papéis das pessoas envolvidas nelas. Nesta seção, apresento uma série de modelos de processo bem genéricos (às vezes chamados de *paradigmas de processo*) partindo de uma perspectiva arquitetural, ou seja, vemos a estrutura do processo, mas não os detalhes de suas atividades.

Esses modelos genéricos são descrições mais gerais e abstratas dos processos de software, e podem ser utilizados para explicar as diferentes abordagens ao desenvolvimento de software. Você pode encará-los como estruturas de processo que podem ser ampliadas e adaptadas para criar processos de engenharia de software mais específicos.

Os modelos de processo genéricos apresentados aqui são:

1. *Modelo em cascata*. Representa as atividades fundamentais do processo, como especificação, desenvolvimento, validação e evolução, na forma de fases de processo distintas, como especificação de requisitos, projeto de software, implementação e testes.
2. *Desenvolvimento incremental*. Intercala as atividades de especificação, desenvolvimento e validação. O sistema é desenvolvido como uma série de versões (incrementos), com cada uma delas acrescentando funcionalidade à versão anterior.
3. *Integração e configuração*. Baseia-se na disponibilidade de componentes ou sistemas reusáveis. O processo de desenvolvimento de sistemas se concentra na configuração desses componentes, para que sejam utilizados em um novo contexto, e na integração deles em um sistema.

Como eu disse, não existe modelo de processo universal aplicável a todos os tipos de desenvolvimento de software. O processo correto depende do cliente e dos

requisitos que regulam o software, do ambiente em que esse software será utilizado e do tipo de software que está sendo desenvolvido. Os sistemas críticos em segurança, por exemplo, são normalmente desenvolvidos a partir de um processo em cascata, já que é necessária uma grande quantidade de análise e de documentação antes de começar sua implementação. Por sua vez, os produtos de software são, atualmente, desenvolvidos a partir de um modelo de processo incremental. Os sistemas de negócio são desenvolvidos, cada vez mais, por meio da configuração dos sistemas preexistentes e da integração entre eles, a fim de criar um novo sistema com a funcionalidade exigida.

Na prática, a maior parte dos processos de software se baseia em um modelo genérico, mas frequentemente incorpora características de outros modelos. Isso vale particularmente para a engenharia dos grandes sistemas. Neles, faz sentido combinar algumas das melhores características de todos os processos genéricos. É preciso ter informações sobre os requisitos de sistema essenciais para projetar uma arquitetura de software que apoie esses requisitos, e não dá para desenvolver isso de modo incremental. Os subsistemas dentro de um sistema maior podem ser desenvolvidos por meio de abordagens diferentes. Partes do sistema que sejam bem compreendidas podem ser especificadas e desenvolvidas usando um processo em cascata ou podem ser adquiridas como sistemas de prateleira para configuração. Outras partes do sistema, difíceis de especificar antecipadamente, devem ser sempre desenvolvidas a partir de uma abordagem incremental. Em ambos os casos, os componentes de software provavelmente serão reusados.

Várias tentativas têm sido feitas para desenvolver modelos de processo “universais” baseados em todos esses modelos genéricos. Um dos mais conhecidos é o *Rational Unified Process* — RUP — (KRUTCHEN, 2003), que foi desenvolvido pela Rational, uma empresa de engenharia de software norte-americana. O RUP é um modelo flexível, que pode ser instanciado de diferentes maneiras para criar processos que se assemelhem a qualquer um dos modelos de processo genéricos discutidos aqui. O RUP foi adotado por algumas grandes empresas de software (principalmente pela IBM), mas não conquistou uma ampla aceitação.

O Rational Unified Process

O *Rational Unified Process* (RUP) reúne os elementos de todos os modelos de processo genéricos discutidos aqui e apoia a prototipação e a entrega incremental do software (KRUTCHEN, 2003). O RUP é descrito normalmente a partir de três perspectivas: uma dinâmica, que mostra as fases do modelo no tempo; uma estática, que mostra as atividades do processo; e uma prática, que sugere práticas a serem utilizadas no processo. As fases do RUP são a concepção, em que se estabelece um *business case* para o sistema; a elaboração, em que são desenvolvidos os requisitos e a arquitetura; a construção, em que o software é implementado; e a transição, em que o sistema é implantado.

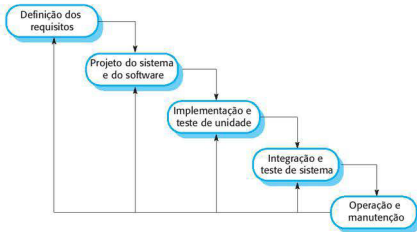


2.1.1 O modelo em cascata

O primeiro modelo de processo de desenvolvimento de software a ser publicado é derivado dos modelos utilizados na engenharia de grandes sistemas militares (ROYCE,

1970). Ele apresenta o processo de desenvolvimento de software como uma série de estágios, conforme a Figura 2.1. Devido à cascata de uma fase para outra, esse modelo é conhecido como modelo em cascata ou ciclo de vida do software. O modelo em cascata é um exemplo de processo dirigido por plano. A princípio, pelo menos, é necessário planejar e criar um cronograma de todas as atividades de processo antes de começar o desenvolvimento do software.

FIGURA 2.1 O modelo em cascata.



Os estágios do modelo em cascata refletem diretamente as atividades fundamentais do desenvolvimento de software:

1. *Análise e definição dos requisitos.* Os serviços, as restrições e as metas do sistema são estabelecidos por meio de consulta aos usuários. Depois, eles são definidos em detalhes e servem como uma especificação do sistema.
2. *Projeto do sistema e do software.* O processo de projeto do sistema reparte os requisitos entre requisitos de sistemas de hardware e de software, e estabelece uma arquitetura global do sistema. O projeto de software envolve a identificação e a descrição das abstrações fundamentais do sistema de software e seus relacionamentos.
3. *Implementação e teste de unidade.* Durante essa etapa, o projeto do software é realizado como um conjunto de programas ou unidades de programa. O teste de unidade envolve a verificação de cada unidade, conferindo se satisfazem a sua especificação.
4. *Integração e teste de sistema.* As unidades de programa ou os programas são integrados e testados como um sistema completo a fim de garantir que os requisitos de software tenham sido cumpridos. Após os testes, o sistema de software é entregue ao cliente.
5. *Operação e manutenção.* Normalmente, essa é a fase mais longa do ciclo de vida. O sistema é instalado e colocado em uso. A manutenção envolve corrigir os erros que não foram descobertos nas primeiras fases do ciclo de vida, melhorar a implementação das unidades do sistema e aperfeiçoar os serviços do sistema à medida que novos requisitos são descobertos.

Modelo de processo espiral de Boehm

Barry Boehm, um dos pioneiros na engenharia de software, propôs um modelo de processo incremental dirigido por riscos. O processo é representado como uma espiral, e não como uma sequência de atividades (BOEHM, 1988).

Cada volta na espiral representa uma fase do processo de software. Desse modo, a volta mais interna estaria relacionada com a viabilidade do sistema; a volta seguinte, com a definição dos requisitos; a próxima, com o projeto (*design*) do sistema, e assim por diante. O modelo em espiral combina prevenção das mudanças com tolerância às mudanças, assumindo que elas são uma consequência dos riscos do projeto, e inclui atividades explícitas de gerenciamento de risco para diminuí-los.



A princípio, o resultado de cada fase no modelo em cascata consiste em um ou mais documentos que são aprovados. A fase seguinte não deve começar até que a fase anterior tenha terminado. No desenvolvimento de hardware, que envolve altos custos de produção, isso faz sentido. No entanto, no desenvolvimento de software, esses estágios se sobrepõem e alimentam uns aos outros com informações. Durante o projeto (*design*), são identificados problemas com os requisitos; durante a codificação, são encontrados problemas com o projeto, e assim por diante. O processo de software, na prática, nunca é um modelo linear simples, pois envolve *feedback* entre as fases.

À medida que surgem novas informações em uma etapa do processo, os documentos produzidos nas etapas anteriores devem ser modificados para refletir as mudanças no sistema. Por exemplo, se for descoberto que um requisito é caro demais para ser implementado, o documento de requisitos deve ser modificado para removê-lo. Entretanto, isso exige a aprovação do cliente, o que implica atraso do processo de desenvolvimento como um todo.

Como consequência, tanto clientes quanto desenvolvedores podem congelar prematuramente a especificação do software para que não sejam feitas outras alterações. Infelizmente, isso significa que os problemas são deixados para depois, ignorados ou contornados por meio de programação. O congelamento prematuro dos requisitos pode significar que o sistema não fará o que o usuário deseja. Também pode levar a sistemas mal estruturados, já que os problemas de projeto (*design*) foram contornados por artifícios de implementação.

Durante a fase final do ciclo de vida (operação e manutenção), o software começa a ser utilizado. Erros e omissões nos requisitos originais do software são descobertos, falhas de programação e de projeto emergem e a necessidade de novas funcionalidades é identificada. Por isso, o sistema deve evoluir a fim de continuar sendo útil. Realizar essas mudanças (manutenção de software) pode envolver a repetição dos estágios de processo prévios.

Na realidade, o software precisa ser flexível e acomodar mudanças à medida que for sendo desenvolvido. A necessidade de comprometimento inicial e retrabalho quando as mudanças são feitas significa que o modelo em cascata é adequado somente para alguns tipos de sistema, tais como:

1. Sistemas embarcados, nos quais o software deve interagir com sistemas de hardware. Em virtude da inflexibilidade do hardware, normalmente não é possível postergar as decisões sobre a funcionalidade do software até que ele seja implementado.
2. Sistemas críticos, nos quais há necessidade de ampla análise da segurança (*safety*) e segurança da informação (*security*) da especificação e do projeto

do software. Nesses sistemas, os documentos de especificação e de projeto devem estar completos para que a análise seja possível. Geralmente, é muito caro corrigir, durante a fase de implementação, os problemas relacionados à segurança na especificação e no projeto.

3. Grandes sistemas de software, que fazem parte de sistemas de engenharia mais amplos, desenvolvidos por várias empresas parceiras. O hardware nos sistemas pode ser desenvolvido a partir de um modelo similar, e as empresas preferem usar um modelo comum para o hardware e o software. Além disso, quando várias empresas estão envolvidas, podem ser necessárias especificações completas para permitir o desenvolvimento independente dos diferentes subsistemas.

O modelo em cascata não é recomendado para situações em que a comunicação informal do time é possível e nas quais os requisitos de software mudam rapidamente. Para esses sistemas, o desenvolvimento iterativo e os métodos ágeis são melhores.

Uma variação importante do modelo em cascata é o desenvolvimento de sistema formal, em que é criado um modelo matemático de uma especificação do sistema. Depois, esse modelo é refinado em código executado, usando transformações matemáticas que preservam sua consistência. Os processos de desenvolvimento formais, como os baseados no método B (ABRIAL, 2005, 2010) são utilizados basicamente no desenvolvimento de sistemas de software que têm requisitos rigorosos de segurança (*safety*), confiabilidade ou segurança da informação (*security*). A abordagem formal simplifica a produção de um caso de segurança (*safety*) ou de segurança da informação (*security*). Isso demonstra aos clientes ou reguladores que o sistema satisfaz na prática os seus requisitos. No entanto, em decorrência dos altos custos de desenvolvimento de uma especificação formal, esse modelo de desenvolvimento raramente é utilizado, exceto na engenharia de sistemas críticos.

2.1.2 Desenvolvimento incremental

O desenvolvimento incremental se baseia na ideia de desenvolver uma implementação inicial, obter *feedback* dos usuários ou terceiros e fazer o software evoluir através de várias versões, até alcançar o sistema necessário (Figura 2.2). As atividades de especificação, desenvolvimento e validação são intercaladas, em vez de separadas, com *feedback* rápido ao longo de todas elas.

FIGURA 2.2 Desenvolvimento incremental.



O desenvolvimento incremental, em alguma de suas formas, é atualmente a abordagem mais comum para o desenvolvimento de aplicações e produtos de software. Essa abordagem pode ser dirigida por plano ou ágil; na maioria das vezes, uma mistura de ambas. Em uma abordagem dirigida por plano, os incrementos do sistema são identificados antecipadamente; se for adotada uma abordagem ágil, os incrementos iniciais são identificados, mas o desenvolvimento dos incrementos finais depende do progresso e das prioridades do cliente.

O desenvolvimento incremental de software, que é parte fundamental dos métodos ágeis, é melhor do que uma abordagem em cascata para os sistemas cujos requisitos estão propensos a mudar durante o processo de desenvolvimento. É o caso da maioria dos sistemas de negócio e dos produtos de software. O desenvolvimento incremental reflete a maneira como solucionamos os problemas: raramente elaboramos uma solução completa para os problemas com antecedência; em vez disso, caminhamos para uma solução em uma série de passos, retrocedendo quando percebemos que cometemos um erro. Ao desenvolver um software de modo incremental, é mais barato e fácil fazer alterações nele durante o processo de desenvolvimento.

Cada incremento ou versão do sistema incorpora parte da funcionalidade necessária para o cliente. Geralmente, os incrementos iniciais incluem a funcionalidade mais importante ou a mais urgente. Isso significa que o cliente ou usuário pode avaliar o sistema em um estágio relativamente precoce no desenvolvimento para ver se ele entrega o que é necessário. Se não entrega, então o incremento atual precisa ser alterado e, possivelmente, uma nova funcionalidade deve ser definida para os incrementos posteriores.

O desenvolvimento incremental tem três grandes vantagens em relação ao modelo em cascata:

1. O custo de implementação das mudanças nos requisitos é reduzido. A quantidade de análise e documentação que precisa ser refeita é significativamente menor do que a necessária ao modelo em cascata.
2. É mais fácil obter *feedback* do cliente sobre o trabalho de desenvolvimento. Os clientes podem comentar as demonstrações de software e ver o quanto foi implementado. Para eles, é mais difícil julgar o progresso a partir dos documentos do projeto (*design*) de software.
3. A entrega e a implantação antecipadas de um software útil para o cliente são possíveis, mesmo se toda a funcionalidade não tiver sido incluída. Os clientes são capazes de usar o software e de obter valor a partir dele mais cedo do que com um processo em cascata.

Problemas do desenvolvimento incremental

Embora o desenvolvimento incremental tenha muitas vantagens, ele não está livre de problemas. A principal dificuldade é o fato de que as grandes organizações têm procedimentos burocráticos que evoluíram ao longo do tempo, o que pode levar a uma incompatibilidade entre esses procedimentos e um processo iterativo ou ágil mais informal.

Às vezes, esses procedimentos existem por um bom motivo. Por exemplo, pode haver procedimentos para garantir que o software satisfaça adequadamente as regulamentações externas (as normas contábeis da lei Sarbanes-Oxley¹ nos Estados Unidos, por exemplo). Como nem sempre é possível mudá-los, os conflitos de processo podem ser inevitáveis.



¹ N. da T.: lei de 2002 que estabeleceu regras mais rígidas de governança corporativa e de controle interno nas empresas.

Do ponto de vista da gestão, a abordagem incremental tem dois problemas:

1. O processo não é visível. Os gerentes precisam de resultados regulares para medir o progresso. Se os sistemas forem desenvolvidos rapidamente, não é econômico produzir documentos que reflitam cada versão do sistema.
2. A estrutura do sistema tende a se degradar à medida que novos incrementos são adicionados. Mudanças regulares deixam o código bagunçado, uma vez que novas funcionalidades são adicionadas de qualquer maneira possível. Fica cada vez mais difícil e caro adicionar novas características a um sistema. Para reduzir a degradação estrutural e a bagunça generalizada no código, os métodos ágeis sugerem que se refatore (melhore e reestruture) o software regularmente.

Os problemas do desenvolvimento incremental se tornam particularmente críticos nos sistemas grandes, complexos e de vida longa, nos quais diferentes times desenvolvem partes distintas do sistema. Os sistemas grandes precisam de um *framework* ou de uma arquitetura estáveis, e as responsabilidades dos diferentes times trabalhando no sistema precisam ser claramente definidas em relação a essa arquitetura. Isso deve ser planejado antecipadamente em vez de desenvolvido de modo incremental.

Adotar o desenvolvimento incremental não significa ter de entregar cada incremento para o cliente. É possível desenvolver um sistema de maneira incremental e expô-lo aos comentários dos clientes e de outros *stakeholders*, sem necessariamente entregá-lo ou implantá-lo no ambiente do cliente. A entrega incremental (coberta na Seção 2.3.2) significa que o software é utilizado em processos operacionais reais, o que faz com que o *feedback* do usuário tenda a ser realista. Entretanto, nem sempre é possível fornecer esse *feedback*, já que a experimentação de um novo software pode atrapalhar os processos normais do negócio.

2.1.3 Integração e configuração

Na maioria dos projetos há algum reuso de software. Com frequência, isso acontece informalmente quando as pessoas que trabalham no projeto conhecem ou procuram algum código similar ao necessário. Elas procuram por esse código, modificam-no conforme a necessidade e integram-no ao novo código que desenvolveram.

Esse reuso informal ocorre independentemente do processo de desenvolvimento utilizado. No entanto, desde os anos 2000, processos de desenvolvimento de software que se concentram no reuso de código existente passaram a ser amplamente utilizados. As abordagens orientadas ao reuso contam com bases de componentes de software reusáveis e um *framework* de integração para a composição desses componentes.

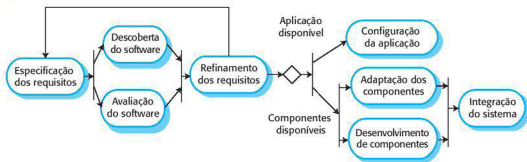
Três tipos de componentes de software são reusados frequentemente:

1. Sistemas de aplicação *stand-alone* configurados para utilização em um ambiente particular. Esses sistemas são de uso geral e possuem muitas características, mas precisam ser adaptados para uso em uma aplicação específica.
2. Coleções de objetos desenvolvidos como um componente ou como um pacote a ser integrado a um *framework* de componentes, como o Java Spring *framework* (WHEELER; WHITE, 2013).
3. *Web services* desenvolvidos de acordo com os padrões de serviço e que estão disponíveis para uso remoto na internet.

A Figura 2.3 mostra um modelo de processo genérico articulado em torno da integração e da configuração para o desenvolvimento de software baseado no reuso. Os estágios nesse processo são:

1. *Especificação dos requisitos.* Os requisitos iniciais do sistema são propostos. Eles não precisam ser elaborados em detalhes, mas devem incluir descrições breves dos requisitos essenciais e das características de sistema desejáveis.
2. *Descoberta e avaliação do software.* Com base em uma descrição dos requisitos de software, é feita uma busca pelos componentes e sistemas que fornecem a funcionalidade necessária. Os candidatos são avaliados para ver se satisfazem os requisitos essenciais e se são genericamente adequados ao uso no sistema.
3. *Refinamento dos requisitos.* Nesse estágio, os requisitos são definidos com base nas informações dos componentes reusáveis e das aplicações que foram descobertas. Os requisitos são modificados para refletir os componentes disponíveis, e a especificação do sistema é redefinida. Onde as modificações forem impossíveis, a atividade da análise de componentes pode ser reintroduzida para procurar soluções alternativas.
4. *Configuração da aplicação.* Se estiver disponível uma aplicação de prateleira que satisfaça os requisitos, ela pode ser configurada para utilização a fim de criar o novo sistema.
5. *Adaptação e integração dos componentes.* Se não houver uma aplicação de prateleira, componentes reusáveis podem ser modificados ou novos componentes podem ser desenvolvidos, visando a integração posterior ao sistema.

FIGURA 2.3 Engenharia de software orientada para o reúso.



A engenharia de software baseada no reúso, articulada em torno da configuração e da integração, tem a vantagem óbvia de reduzir a quantidade de software a ser desenvolvido, diminuindo custos e riscos. Normalmente, isso também leva a uma entrega mais rápida do software. Entretanto, concessões quanto aos requisitos são inevitáveis, o que pode resultar em um sistema que não satisfaça as necessidades reais dos usuários. Além disso, parte do controle sobre a evolução do sistema se perde, já que novas versões dos componentes reusáveis não estão sob o controle da organização que os utiliza.

O reúso de software é muito importante e, portanto, vários capítulos na terceira parte deste livro foram dedicados ao tema. As questões gerais de reúso de software são abordadas no Capítulo 15; a engenharia de software baseada em componentes, nos Capítulos 16 e 17; os sistemas orientados a serviços, no Capítulo 18.

2.2 ATIVIDADES DO PROCESSO

Os processos de software reais são sequências intercaladas de atividades técnicas, colaborativas e gerenciais, cujo objetivo global é especificar, projetar, implementar e testar um sistema de software. Geralmente, os processos são apoiados por

ferramentas. Isso significa que os desenvolvedores de software podem usar uma gama de ferramentas de software para ajudá-los, como sistemas de gerenciamento de requisitos, editores de modelo de projeto (*design*), editores de programa, ferramentas de teste automatizadas e depuradores.

As quatro atividades de processo básicas — a especificação, o desenvolvimento, a validação e a evolução — são organizadas de modo distinto em diferentes processos de desenvolvimento. No modelo em cascata, elas são organizadas em sequência; no desenvolvimento incremental, são intercaladas. O modo como essas atividades são executadas depende do tipo de software que está sendo desenvolvido, da experiência e da competência dos desenvolvedores e do tipo de empresa que o desenvolve.

Ferramentas de desenvolvimento de software

Ferramentas de desenvolvimento de software são programas utilizados para apoiar as atividades do processo de engenharia de software e incluem ferramentas de gerenciamento de requisitos, editores de projeto, ferramentas de apoio à refatoração, compiladores, depuradores, rastreadores de defeitos (*bug trackers*) e ferramentas de construção de sistemas.

As ferramentas de software fornecem suporte ao processo ao automatizarem algumas de suas atividades e ao fornecerem informações sobre o software que está sendo desenvolvido. Por exemplo:

- ▶ O desenvolvimento de modelos gráficos do sistema como parte da especificação de requisitos ou do projeto (*design*) de software.
- ▶ A geração de código a partir desses modelos gráficos.
- ▶ A geração de interfaces com o usuário a partir de uma descrição da interface gráfica criada interativamente por esse usuário.
- ▶ Depuração de programas por meio do fornecimento de informações a respeito de um programa em execução.
- ▶ A tradução automática para uma versão mais recente dos programas escritos com versões antigas de uma linguagem de programação.



As ferramentas podem ser combinadas em um arcabouço chamado de ambiente de desenvolvimento integrado (IDE, do inglês *Integrated Development Environment*), que fornece um conjunto comum de recursos que as ferramentas podem utilizar; assim, tanto a comunicação como a operação de maneira integrada são facilitadas.

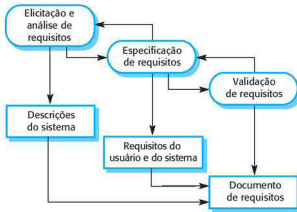
2.2.1 Especificação do software

Especificação do software ou engenharia de requisitos é o processo de compreender e definir quais serviços são necessários para o sistema e identificar as restrições sobre sua operação e desenvolvimento. A engenharia de requisitos é um estágio particularmente crítico do processo de software, já que os erros cometidos nessa etapa inevitavelmente geram problemas posteriores no projeto e na implementação do sistema.

Antes de iniciar o processo de engenharia dos requisitos, uma empresa pode realizar um estudo de viabilidade ou de marketing para avaliar se há ou não uma demanda ou um mercado para o software e se ele é realista ou não em termos técnicos e financeiros. Os estudos de viabilidade são de curto prazo, relativamente baratos e orientam a decisão de ir adiante ou não com uma análise mais detalhada.

O processo de engenharia de requisitos (Figura 2.4) visa à produção de um documento de requisitos acordados que especifique um sistema que satisfaça os requisitos dos *stakeholders*. Os requisitos são apresentados normalmente em dois níveis de detalhe. Os usuários finais e os clientes precisam de uma declaração de requisitos mais superficial desse sistema; os desenvolvedores, de uma especificação mais detalhada.

FIGURA 2.4 O processo de engenharia de requisitos.



Existem três atividades principais no processo de engenharia de requisitos:

1. *Elicitação e análise de requisitos.* É o processo de derivação dos requisitos do sistema por meio da observação dos sistemas existentes, de discussões com os potenciais usuários e clientes, da análise de tarefas etc. Pode envolver o desenvolvimento de um ou mais modelos do sistema e protótipos, pois eles ajudam a compreender o sistema a ser especificado.
2. *Especificação de requisitos.* É a atividade de traduzir a informação obtida durante a análise em um documento que defina um conjunto de requisitos. Dois tipos podem ser incluídos nesse documento: requisitos do usuário, que são declarações abstratas dos requisitos do sistema para o cliente e usuário final; e requisitos do sistema, que são uma descrição mais detalhada da funcionalidade a ser fornecida.
3. *Validação de requisitos.* Essa atividade confere os requisitos quanto a realismo, consistência e integridade. Durante esse processo, erros no documento de requisitos são inevitavelmente descobertos. Assim, o documento deve ser modificado para corrigir tais problemas.

A análise de requisitos prossegue durante as atividades de definição e de especificação, e novos requisitos surgem nesse processo. Portanto, as atividades de análise, definição e especificação estão entrelaçadas.

Nos métodos ágeis, a especificação de requisitos não é uma atividade separada, mas parte do desenvolvimento do sistema. Os requisitos são especificados informalmente para cada incremento do sistema imediatamente antes de ele ser desenvolvido. Os requisitos são especificados de acordo com as prioridades do usuário, e sua elicitação vem dos usuários que fazem parte ou que trabalham em estreita colaboração com o time de desenvolvimento.

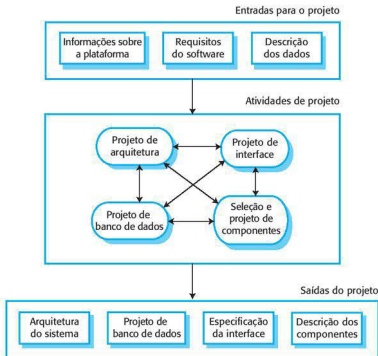
2.2.2 Projeto e implementação do software

O estágio de implementação no desenvolvimento de software é o processo de elaborar um sistema executável para ser entregue ao cliente. Às vezes, isso envolve atividades distintas, que são o projeto (*design*) e a programação do software. No entanto, se uma abordagem ágil for utilizada para o desenvolvimento, o projeto e a implementação são intercalados, sem documentos de projeto (*design*) formais produzidos durante esse processo. Naturalmente, o software ainda é projetado, mas o projeto está registrado informalmente nas lousas ou nas anotações feitas pelos programadores.

O projeto de software é uma descrição da estrutura do software a ser implementado, dos modelos e estruturas de dados utilizados pelo sistema, das interfaces entre os componentes do sistema e, às vezes, do algoritmo utilizado. Os projetistas não chegam a um projeto acabado imediatamente, mas o desenvolvem em estágios. Eles acrescentam detalhes à medida que desenvolvem seu projeto, com revisões constantes para modificar os projetos iniciais.

A Figura 2.5 é um modelo abstrato do processo de projeto, mostrando suas entradas, suas atividades e suas saídas. As atividades do processo de projeto são intercaladas e interdependentes. Novas informações sobre ele estão sendo geradas constantemente, o que afeta as decisões de projeto anteriores. O retrabalho é, portanto, inevitável.

FIGURA 2.5 Um modelo geral do processo de projeto.



A maioria dos softwares interage com outros sistemas de software, incluindo desde o sistema operacional e bancos de dados até *middleware* e outras aplicações. Tudo isso compõe a "plataforma de software", o ambiente no qual o software será executado. As informações sobre essa plataforma são a entrada essencial para o processo de projeto, pois os projetistas poderão decidir melhor como integrar o

software ao seu ambiente. Se o sistema tiver de processar dados existentes, então a descrição desses dados poderá ser incluída na especificação da plataforma. Caso contrário, essa descrição deverá ser uma entrada para o processo de projeto, para que a organização dos dados do sistema possa ser definida.

As atividades no processo de projeto variam, dependendo do tipo de sistema que está sendo desenvolvido. Por exemplo, os sistemas de tempo real necessitam de um estágio adicional de projeto de sincronismo, mas podem não incluir um banco de dados, então não há um projeto de banco de dados envolvido. A Figura 2.5 mostra quatro atividades que podem fazer parte do processo de projeto para sistemas de informação:

1. *Projeto de arquitetura*, em que são identificados a estrutura global do sistema e os componentes principais (às vezes chamados de subsistemas ou módulos), observando seus relacionamentos e como eles estão distribuídos.
2. *Projeto de banco de dados*, em que são projetadas as estruturas de dados do sistema e como elas devem ser representadas em um banco de dados. O trabalho aqui depende da definição entre reusar um banco de dados ou criar um novo.
3. *Projeto de interface*, em que são definidas as interfaces entre os componentes do sistema; essa especificação de interfaces deve ser inequívoca. Com uma interface precisa, um componente pode ser utilizado por outros sem que seja preciso saber como ele é implementado. Uma vez acordadas as especificações da interface, os componentes podem ser projetados e desenvolvidos separadamente.
4. *Seleção e projeto de componentes*, em que são feitas buscas por componentes reusáveis e, caso não haja componentes adequados, são projetados novos componentes de software. O projeto, nesse estágio, pode ser uma descrição simples dos componentes, com os detalhes de implementação deixados para o programador. Como alternativa, pode ser uma lista de alterações a serem feitas em um componente reusável ou um modelo de projeto detalhado expresso em UML. O modelo de projeto pode ser utilizado para gerar automaticamente uma implementação.

Essas atividades levam às saídas do projeto, que também são exibidas na Figura 2.5. Nos sistemas críticos, os resultados do processo de projeto são documentos detalhados contendo descrições precisas do sistema. Se uma abordagem dirigida por modelo (Capítulo 5) for utilizada, as saídas do projeto serão diagramas. Se métodos ágeis forem utilizados, as saídas do processo de projeto não serão documentos de especificação separados, mas estarão representadas no código do programa.

O desenvolvimento de um programa para implementar um sistema é o próximo passo natural do projeto. Embora algumas classes de programa, como os sistemas críticos em segurança, normalmente sejam projetadas em detalhes antes de qualquer implementação, é mais comum que o projeto e a programação sejam intercalados. Há ferramentas de desenvolvimento de software que podem ser usadas para gerar um esqueleto de programa a partir de um projeto. Isso inclui o código para definir e implementar interfaces e, em muitos casos, o desenvolvedor precisará apenas acrescentar detalhes da operação de cada componente do programa.

A programação é uma atividade individual e não existe um processo genérico que seja seguido habitualmente. Alguns programadores começam desenvolvendo componentes que eles entendem e depois passam a outros componentes menos conhecidos; outros adotam a abordagem oposta, deixando os componentes

familiares para o fim, pois já sabem como desenvolvê-los. Alguns desenvolvedores gostam de definir os dados logo no início do processo e depois usá-los para conduzir o desenvolvimento dos programas; outros ficam o máximo de tempo possível sem especificar os dados.

Normalmente, os programadores realizam algum tipo de teste do código que desenvolveram. Frequentemente isso revela defeitos do programa (*bugs*) que devem ser removidos. A atividade de encontrar e corrigir os defeitos do programa é chamada de depuração (*debugging*). Os testes e a depuração dos defeitos são processos diferentes. Os testes estabelecem a existência dos defeitos. A depuração está relacionada com a localização e a correção deles.

A depuração deve gerar hipóteses a respeito do comportamento observável do programa e depois testá-las com o objetivo de encontrar o defeito que causou o resultado anormal. Testar as hipóteses pode envolver rastrear o código do programa manualmente, o que pode exigir novos casos de teste para localizar o problema. Para apoiar esse processo, normalmente são utilizadas ferramentas de depuração interativa, que exibem os valores intermediários das variáveis do programa e um rastro (*trace*) dos comandos executados.

2.2.3 Validação do software

A validação do software ou, em termos mais gerais, verificação e validação (V & V), destina-se a mostrar que um sistema está em conformidade com sua especificação e que satisfaz as expectativas do cliente do sistema. A principal técnica de validação é o teste de programa, no qual o sistema é executado usando dados de teste simulados. A validação também pode envolver processos de conferência como inspeções e revisões em cada estágio do processo de software, desde a definição dos requisitos do usuário até o desenvolvimento do programa. Entretanto, a maior parte do tempo e esforço de V & V é consumida no teste do programa.

Exceto no caso de programas pequenos, os sistemas não devem ser testados como uma unidade monolítica. A Figura 2.6 mostra um processo de teste em três estágios, no qual os componentes do sistema são testados individualmente e, depois, o sistema integrado é testado. No software personalizado, o teste do cliente envolve testar o sistema com dados reais do cliente. Nos produtos vendidos como aplicações, o teste do cliente é feito por usuários selecionados, que experimentam e comentam o software, o que é conhecido como teste-beta.

FIGURA 2.6 Estágios do teste.



Os estágios no processo de teste são:

1. *Teste de componente.* Os componentes do sistema são testados pelas pessoas que o desenvolvem. Cada componente é testado independentemente, sem as demais partes do sistema. Os componentes podem ser entidades simples,

como as funções ou classes de objetos, ou agrupamentos coerentes dessas entidades. Ferramentas de automação dos testes, como a JUnit para Java, que podem reexecutar testes quando são criadas novas versões do componente, são frequentemente utilizadas (KOSKELA, 2013).

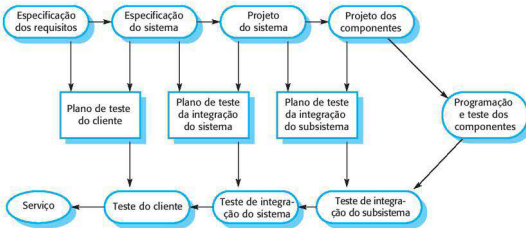
2. *Teste de sistema.* Os componentes do sistema são integrados para criar um sistema completo. Esse processo encontra erros resultantes de interações imprevistas entre os componentes e de problemas de interface. Também busca mostrar que o sistema satisfaz tanto requisitos funcionais quanto não funcionais e testa suas propriedades emergentes. Nos sistemas grandes, esse processo pode ter várias etapas, nas quais os componentes são integrados e formam subsistemas testados individualmente antes de serem integrados ao sistema final.
3. *Teste do cliente.* Esse é o estágio final no processo de teste antes de o sistema ser aceito para uso operacional. O sistema é testado pelo cliente (ou cliente potencial) em vez de usar dados de simulação. No software criado por encomenda, o teste do cliente pode revelar erros e omissões na definição dos requisitos do sistema, pois os dados reais exercitam o sistema de maneiras diferentes das que ocorrem com os dados de teste. O teste do cliente também pode revelar problemas de requisitos nos quais os recursos do sistema não satisfazem realmente as necessidades dos usuários ou o desempenho do sistema é inaceitável. Nos produtos, o teste do cliente mostra o quanto o produto de software satisfaz as necessidades do cliente.

Em condições ideais, os defeitos de componentes são descobertos cedo no processo de teste, e os problemas de interface, quando o sistema é integrado. No entanto, à medida que os defeitos são descobertos, o programa deve ser depurado e isso pode exigir que outros estágios no processo de teste sejam repetidos. Os erros nos componentes do programa podem aparecer durante o teste do sistema. Portanto, o processo é iterativo, com as informações sendo retroalimentadas dos estágios finais para as partes iniciais do processo.

Normalmente, o teste de componentes é uma simples parte do processo de desenvolvimento normal. Os programadores produzem seus próprios dados de teste e testam o código de modo incremental à medida que o desenvolvem. O programador conhece o componente e, portanto, é a melhor pessoa para gerar os casos de teste.

Se for utilizada uma abordagem incremental de desenvolvimento, cada incremento deve ser testado enquanto é desenvolvido, e os testes devem ser baseados nos requisitos para aquele incremento. No desenvolvimento dirigido por testes, que é uma parte normal dos processos ágeis, os testes são desenvolvidos junto com os requisitos, antes do início do desenvolvimento. Isso ajuda os testadores e os desenvolvedores a compreenderem os requisitos e garante que não haja atrasos enquanto os casos de teste são criados.

Quando um processo de software dirigido por plano é utilizado (no desenvolvimento de sistemas críticos, por exemplo), o teste é dirigido por um conjunto de planos de teste. Uma equipe independente de testadores trabalha seguindo esses planos de teste que foram desenvolvidos a partir da especificação e do projeto do sistema. A Figura 2.7 ilustra como os planos de teste funcionam como elo entre as atividades de teste e as de desenvolvimento. É o que alguns chamam de “modelo em V”, que mostra quais atividades de validação do software correspondem a cada estágio do modelo de processo em cascata.

FIGURA 2.7 Fases de teste em um processo de software dirigido por plano.

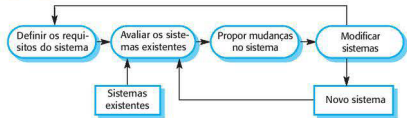
Quando um sistema é comercializado como um produto de software, geralmente se usa um processo de teste conhecido como teste-beta. Esse teste envolve a entrega do sistema para uma série de possíveis clientes que concordam em utilizá-lo experimentalmente, a fim de relatar problemas para os desenvolvedores do sistema. Esse procedimento expõe o produto ao uso real e detecta erros que podem não ter sido previstos pelos desenvolvedores do produto. Após esse *feedback*, o produto de software pode ser modificado e liberado para mais testes-beta ou para a venda geral.

2.2.4 Evolução do software

A flexibilidade do software é uma das principais razões pelas quais, cada vez mais, ele é incorporado a sistemas grandes e complexos. Depois de tomada a decisão de produzir o hardware, é muito caro fazer alterações em seu projeto (*design*). Entretanto, alterações no software podem ser feitas a qualquer momento, durante ou depois do desenvolvimento do sistema. Mesmo as grandes mudanças ainda são muito mais baratas do que mudanças equivalentes no hardware do sistema.

Historicamente, sempre houve uma divisão entre o processo de desenvolvimento e o processo de evolução (manutenção) do software. As pessoas pensam em desenvolvimento de software como uma atividade criativa, na qual um sistema de software é desenvolvido a partir de um conceito inicial até se tornar um sistema funcional. Por outro lado, pensam em manutenção de software como uma atividade maçante, menos interessante e menos desafiadora do que o desenvolvimento do software original.

Essa distinção entre desenvolvimento e manutenção é cada vez mais irrelevante. Poucos sistemas de software são completamente novos, e faz muito mais sentido encarar o desenvolvimento e a manutenção como uma coisa só. Em vez de processos diferentes, é mais realista encarar a engenharia de software como um processo evolutivo (Figura 2.8), no qual o software é alterado continuamente ao longo de sua vida útil em resposta à mudança dos requisitos e das necessidades do cliente.

FIGURA 2.8 Evolução do sistema de software.

2.3 LIDANDO COM MUDANÇAS

A mudança é inevitável em todos os grandes projetos de software. Os requisitos do sistema mudam à medida que as empresas reagem a pressões externas, à concorrência e a mudanças nas prioridades da gestão. Ao passo que novas tecnologias são disponibilizadas, novas abordagens de projeto e de implementação se tornam possíveis. Portanto, seja qual for o modelo de processo de software utilizado, é essencial que ele consiga apoiar as mudanças no software que está sendo desenvolvido.

A mudança eleva os custos de desenvolvimento de software, já que isso normalmente significa que o trabalho já concluído precisará ser refeito: isso é retrabalho. Por exemplo, se os relacionamentos entre os requisitos em um sistema forem analisados e novos requisitos forem identificados, parte ou toda a análise de requisitos deve ser refeita. Então, pode ser necessário reprojeter o sistema para entregar os novos requisitos, mudar quaisquer programas que tenham sido desenvolvidos e testar o sistema novamente.

Duas abordagens relacionadas podem ser utilizadas para reduzir os custos de retrabalho:

1. **Antecipação da mudança.** O processo de software inclui atividades que podem antecipar ou prever possíveis mudanças antes da necessidade de um retrabalho considerável. Por exemplo, um protótipo do sistema pode ser desenvolvido para exibir aos clientes algumas características principais do sistema. Eles podem experimentar o protótipo e refinar seus requisitos antes de se comprometerem com os altos custos de produção do software.
2. **Tolerância à mudança.** O processo e o software são projetados de modo que as mudanças no sistema possam ser feitas com facilidade. Isso envolve, normalmente, alguma forma de desenvolvimento incremental. As mudanças propostas podem ser implementadas em incrementos que ainda não foram desenvolvidos. Se isso for impossível, então apenas um único incremento (uma pequena parte do sistema) pode precisar de alteração a fim de incorporar a mudança.

Nesta seção, discuto duas maneiras de lidar com as mudanças e com as variações nos requisitos do sistema:

1. **Prototipação do sistema.** Uma versão ou parte do sistema é desenvolvida rapidamente para verificar os requisitos do cliente e a viabilidade de algumas decisões de projeto. Essa é uma maneira de antecipar a mudança, já que permite aos usuários experimentarem o sistema antes da entrega e, assim, refinar seus

requisitos. Como consequência, a quantidade de propostas de alteração nos requisitos feitas após a entrega tende a ser reduzida.

2. *Entrega incremental.* O sistema é fornecido para o cliente em incrementos, a fim de que comentários e experimentações sejam feitos. Essa é uma maneira de antecipar as mudanças e aumentar a tolerância a elas, evitando o comprometimento prematuro com os requisitos do sistema como um todo e permitindo que as mudanças sejam incorporadas aos incrementos finais a um custo relativamente baixo.

O conceito de refatoração — ou seja, a melhoria da estrutura e da organização de um programa — também é um importante mecanismo de suporte de tolerância à mudança. Discutirei esse tema no Capítulo 3.

2.3.1 Prototipação

O protótipo é uma versão inicial de um sistema utilizado para demonstrar conceitos, experimentar opções de projeto e descobrir mais sobre o problema e suas possíveis soluções. O desenvolvimento rápido e iterativo do protótipo é essencial para que os custos sejam controlados e os *stakeholders* do sistema possam experimentar o protótipo no início do processo de desenvolvimento do software.

Um protótipo de software pode ser utilizado em um processo de desenvolvimento para ajudar a antecipar as mudanças que podem ser necessárias:

1. No processo de engenharia de requisitos, um protótipo pode ajudar na elicitación e validação dos requisitos do sistema.
2. No processo de projeto do sistema, um protótipo pode ser utilizado para explorar soluções de software e no desenvolvimento de uma interface com o usuário para o sistema.

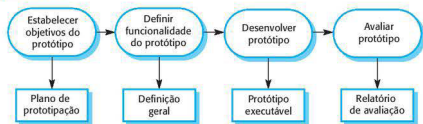
Os protótipos de sistema permitem que usuários em potencial observem até que ponto o sistema os ajuda em seu trabalho; esses usuários podem ter novas ideias a partir dos requisitos e encontrar pontos fortes e fracos no software para, então, propor novos requisitos de sistema. Além disso, à medida que o protótipo é desenvolvido, ele pode revelar erros e omissões nesses requisitos. Uma característica descrita na especificação pode parecer clara e útil; no entanto, quando essa função é combinada com outras, muitas vezes os usuários acham que a sua opinião inicial estava errada ou incompleta. A especificação do sistema pode ser modificada para refletir a mudança na compreensão dos requisitos.

Um protótipo do sistema pode ser utilizado — enquanto o próprio sistema estiver sendo projetado — para experimentos que visem averiguar a viabilidade do projeto proposto. Por exemplo: um projeto de banco de dados pode ser prototipado e testado para averiguar se ele suporta de maneira eficiente os acessos aos dados gerados pelas requisições mais comuns dos usuários. A prototipação rápida com envolvimento do usuário final é a única maneira coerente de desenvolver interfaces com o usuário. Devido à natureza dinâmica das interfaces com o usuário, as descrições textuais e os diagramas não são suficientemente bons para expressar o projeto e os requisitos de uma interface com o usuário.

Um modelo de processo para o desenvolvimento de um protótipo é exibido na Figura 2.9. Os objetivos da prototipação devem ser explicitados desde o início do

processo. Esses objetivos podem ser o desenvolvimento da interface com o usuário, o desenvolvimento de um sistema para validar os requisitos funcionais ou o desenvolvimento de um sistema para demonstrar a aplicação para os gerentes. Geralmente, o mesmo protótipo não consegue cumprir todos os objetivos. Se os objetivos não forem declarados, a gestão ou os usuários finais podem entender mal a função do protótipo. Consequentemente, eles podem não obter os benefícios que esperavam do desenvolvimento do protótipo.

FIGURA 2.9 Desenvolvimento de um protótipo.

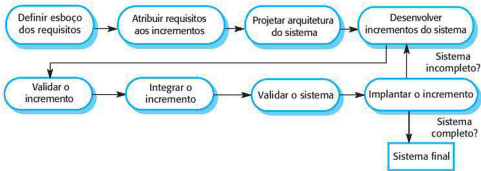


O próximo estágio no processo é decidir o que colocar e, talvez ainda mais importante, o que deixar de fora do sistema prototipado. Para reduzir os custos de prototipação e acelerar o cronograma de entrega, é possível deixar parte da funcionalidade fora do protótipo ou afrouxar os requisitos não funcionais, como o tempo de resposta e a utilização de memória. O tratamento e o gerenciamento dos erros podem ser ignorados, a menos que o objetivo do protótipo seja o de estabelecer uma interface com o usuário. Os padrões de confiabilidade e qualidade do programa podem ser reduzidos.

O estágio final do processo é a avaliação do protótipo. Nessa etapa, deve ser realizado o treinamento dos usuários, e os objetivos do protótipo devem ser usados para a criação de um plano de avaliação. Os usuários precisam de tempo para se acostumar com um sistema novo e estabelecer um padrão normal de uso. Isso feito, é possível descobrir erros e omissões nos requisitos. Um problema geral com a prototipação é que os usuários podem não utilizar o protótipo da mesma maneira que utilizam o sistema final. Os testadores do protótipo podem não ser usuários típicos do sistema. Pode não haver tempo suficiente para treinar os usuários durante a avaliação do protótipo. Se o protótipo for lento, os avaliadores poderão ajustar sua maneira de trabalhar, a fim de evitar as características do sistema com tempos de resposta lentos; quando receberem uma resposta melhor, no sistema final, eles poderão utilizá-lo de uma maneira diferente.

2.3.2 Entrega incremental

A entrega incremental (Figura 2.10) é uma abordagem para o desenvolvimento de software em que alguns dos incrementos desenvolvidos são fornecidos para o cliente e implantados para uso em seu ambiente de trabalho. Em um processo de entrega incremental, os clientes definem quais serviços são mais e menos importantes para eles. Uma série de incrementos é definida para entrega, e cada um deles proporciona um subconjunto da funcionalidade do sistema. A alocação dos serviços aos incrementos depende da prioridade do serviço; são implementados e entregues, em primeiro lugar, os de maior prioridade.

FIGURA 2.10 Entrega incremental.

Uma vez que os incrementos do sistema tenham sido identificados, os requisitos dos serviços a serem entregues no primeiro incremento são definidos em detalhes e desenvolvidos. Durante o desenvolvimento, pode ocorrer outra análise de requisitos dos incrementos posteriores, mas mudanças nos requisitos do incremento atual não são aceitas.

Depois que um incremento é concluído e entregue, ele é instalado no ambiente de trabalho normal do cliente, que pode experimentá-lo; isso o ajudará a esclarecer os requisitos dos incrementos posteriores. À medida que novos incrementos são concluídos, eles são integrados aos existentes de modo a melhorar a funcionalidade do sistema a cada incremento fornecido.

A entrega incremental tem uma série de vantagens:

1. Os clientes podem usar incrementos iniciais como protótipos e adquirir experiência, que servirá para informar seus requisitos para incrementos posteriores do sistema. Ao contrário dos protótipos, os incrementos fazem parte do sistema real, então não haverá reaprendizagem quando o sistema completo estiver disponível.
2. Os clientes não precisam esperar até o sistema inteiro ser fornecido para tirar vantagem dele. O primeiro incremento satisfaz suas demandas mais críticas, o que permite usar o software imediatamente.
3. O processo mantém os benefícios do desenvolvimento incremental, posto que deve ser relativamente fácil incorporar mudanças no sistema.
4. Como os serviços de prioridade mais elevada são fornecidos primeiro e os incrementos posteriores são integrados, os serviços de sistema mais importantes recebem a maioria dos testes. Isso significa que os clientes estarão menos propensos a encontrar falhas de software nas partes mais importantes do sistema.

No entanto, também existem problemas na entrega incremental. Na prática, ela funciona apenas nas situações em que um sistema novíssimo está sendo introduzido e os avaliadores do sistema têm tempo para experimentá-lo. Os problemas fundamentais dessa abordagem são:

1. A entrega iterativa é problemática quando o novo sistema substituirá um preexistente. Os usuários precisam de toda a funcionalidade do sistema antigo e, normalmente, não estão dispostos a experimentar um novo sistema que esteja incompleto. Muitas vezes é impraticável usar ambos os sistemas lado a lado, já que eles tendem a ter bancos de dados e interfaces com o usuário diferentes.

2. A maioria dos sistemas requer um conjunto de recursos básicos que são utilizados por diferentes partes do sistema. Como os requisitos não são definidos em detalhes até que um incremento possa ser implementado, pode ser difícil identificar os recursos comuns necessários para todos os incrementos.
3. A essência dos processos iterativos é que a especificação é desenvolvida em conjunto com o software. No entanto, isso entra em conflito com o modelo de aquisição de muitas organizações, em que a especificação completa do sistema faz parte de seu contrato de desenvolvimento. Na abordagem incremental, não há especificação completa do sistema até o incremento final ser especificado. Isso requer uma nova forma de contrato, à qual grandes clientes, a exemplo de órgãos governamentais, podem encontrar dificuldade para se adaptar.

Para alguns tipos de sistemas, o modelo incremental de desenvolvimento e de entrega não é a melhor abordagem. Alguns são sistemas muito grandes, nos quais o desenvolvimento pode envolver times trabalhando em diferentes locais; outros são embarcados, nos quais o software depende do desenvolvimento do hardware; e alguns são críticos, nos quais todos os requisitos devem ser analisados para conferir as interações que podem comprometer a segurança (*safety*) ou a segurança da informação (*security*) do sistema.

Tais sistemas, naturalmente, sofrem com os mesmos problemas de requisitos incertos e mutáveis. Portanto, para solucionar esses problemas e obter alguns dos mesmos benefícios do desenvolvimento incremental, um protótipo do sistema pode ser desenvolvido e utilizado como uma plataforma de experimentação com os requisitos e o projeto do sistema. Com a experiência adquirida no protótipo, os requisitos definitivos poderão ser decididos.

2.4 MELHORIA DE PROCESSO

Hoje em dia, há uma demanda constante da indústria por software melhor e mais barato, que deve ser fornecido em prazos ainda mais apertados. Consequentemente, muitas empresas se voltaram para a melhoria do processo de software como uma maneira de melhorar a qualidade de seu software, reduzindo custos ou acelerando seus processos de desenvolvimento. A melhoria dos processos significa compreender os processos existentes e modificá-los para aumentar a qualidade do produto e/ou reduzir os custos e o tempo de desenvolvimento. Abordo com mais detalhes as questões gerais de medição e melhoria dos processos no Capítulo 26, disponível no site de apoio (em inglês).

São empregadas duas abordagens bem diferentes para melhoria e mudança de processos:

1. A abordagem de maturidade do processo, que se concentra na melhoria dos processos e do gerenciamento do projeto e na introdução de práticas recomendadas de engenharia de software em uma organização. O nível de maturidade do processo reflete o grau em que as práticas técnicas e gerenciais recomendadas foram adotadas nos processos de desenvolvimento de software organizacional. Os objetivos primários dessa abordagem são a maior qualidade do produto e a previsibilidade do processo.

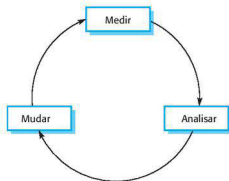
2. A abordagem ágil, que se concentra no desenvolvimento iterativo e na redução dos custos gerais do processo de software. As características primárias dos métodos ágeis são a entrega rápida da funcionalidade e a rapidez de resposta para os requisitos mutáveis do cliente. Aqui, a filosofia da melhoria é que os melhores processos são aqueles com custos gerais mais baixos, e as abordagens ágeis podem conseguir isso. Descrevo os métodos ágeis no Capítulo 3.

Em geral, os entusiastas dessas abordagens e as pessoas comprometidas com cada uma delas são céticas quanto aos benefícios da outra. A abordagem de maturidade do processo está arraigada no desenvolvimento dirigido por plano e, normalmente, requer “custos” mais elevados no sentido de que são introduzidas atividades que não são diretamente relevantes para o desenvolvimento do programa. As abordagens ágeis se concentram no código que está sendo desenvolvido e minimizam deliberadamente a formalidade e a documentação.

A melhoria geral do processo subjacente à abordagem de maturidade do processo é cíclica, como mostra a Figura 2.11. Os estágios nesse processo são:

1. *Medição do processo.* Um ou mais atributos do processo ou do produto de software são medidos. Essas medições formam um ponto de partida que permite decidir se as melhorias do processo foram eficazes. À medida que se introduzem mais melhorias, os mesmos atributos devem ser reavaliados, uma vez que se espera que tenha havido alguma alteração positiva.
2. *Análise do processo.* O processo atual é avaliado a fim de identificar os pontos fracos e os gargalos. Os modelos de processo (às vezes chamados mapas de processo) que o descrevem podem ser desenvolvidos durante esse estágio. A análise pode ser focada no considerar características do processo como rapidez e robustez.
3. *Mudança no processo.* As mudanças são propostas para abordar alguns dos pontos fracos identificados. Elas são introduzidas e o ciclo continua coletando dados sobre sua eficácia.

FIGURA 2.11 O ciclo de melhoria do processo.



Sem dados concretos sobre um processo ou sobre o software desenvolvido é impossível avaliar o valor da melhoria do processo. No entanto, é improvável que as empresas que começam uma mudança em um processo tenham dados disponíveis sobre ele, a fim de determinar um ponto de partida para essa melhoria. Portanto, como parte do primeiro ciclo de mudanças, pode ser necessário coletar dados sobre o processo de software e medir as características do produto.

A melhoria dos processos é uma atividade de longo prazo; então, cada um dos estágios pode durar vários meses. Também é uma atividade contínua, já que, sempre que novos processos são introduzidos, o ambiente de negócios muda e novos processos terão de evoluir para levar em conta essas alterações.

O conceito de “maturidade do processo” foi introduzido no final dos anos 1980 quando o *Software Engineering Institute*² (SEI) propôs seu modelo de maturidade da capacidade do processo (HUMPHREY, 1988). A maturidade dos processos de uma empresa de software reflete seu gerenciamento de processos, sua medição e o uso que ela faz de práticas recomendadas em engenharia de software. Essa ideia foi introduzida para que o Departamento de Defesa dos Estados Unidos pudesse avaliar a capacidade de engenharia de software dos empreiteiros do setor de defesa, com vistas a limitar os contratos àqueles empreiteiros que atingissem um nível exigido de maturidade do processo. Cinco níveis de maturidade do processo foram propostos, conforme a Figura 2.12. Eles evoluíram e se desenvolveram ao longo dos últimos 25 anos (CHRISSIS; KONRAD; SHRUM, 2011), mas as ideias fundamentais do modelo de Humphrey ainda são a base da avaliação da maturidade dos processos de software.

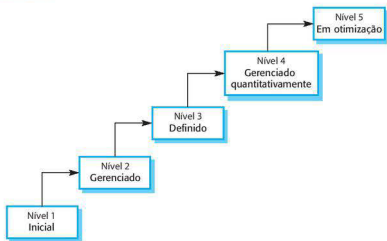
Os níveis do modelo de maturidade do processo são:

1. *Inicial*. As metas associadas à área de processo são cumpridas e o escopo do trabalho a ser realizado é definido explicitamente para todos os processos e comunicado para os membros do time.
2. *Gerenciado*. Nesse nível, as metas associadas à área de processo são cumpridas e as políticas organizacionais estão implantadas, definindo quando cada processo deve ser utilizado. Deve haver planos de projeto documentados que definam as metas do projeto. O gerenciamento de recursos e o monitoramento de processos devem estar em vigor em toda a instituição.
3. *Definido*. Esse nível se concentra na padronização organizacional e na implantação dos processos. Cada projeto tem um processo gerido que é adaptado aos requisitos do projeto a partir de um conjunto definido de processos organizacionais. Os recursos do processo e suas medições devem ser coletados e utilizados para melhorias futuras.
4. *Gerenciado quantitativamente*. Nesse nível, há uma responsabilidade organizacional de empregar o método estatístico — entre outros métodos quantitativos — para controlar os subprocessos. Em outras palavras, as medições de processos e produtos coletadas devem ser utilizadas no gerenciamento dos processos.
5. *Em otimização*. Nesse nível mais alto, a organização deve usar as medições de processos e produtos para orientar a melhoria dos processos. As tendências devem ser analisadas e os processos adaptados às necessidades mutáveis da empresa.

O trabalho sobre os níveis de maturidade do processo teve um impacto importante na indústria de software. Ele concentrou a atenção nos processos e práticas utilizados e proporcionou melhorias significativas na capacidade da engenharia de software. No entanto, a melhoria do processo formal para empresas pequenas é muito custosa e a estimativa da maturidade com os processos ágeis é difícil. Consequentemente, apenas as grandes empresas de software usam hoje essa abordagem focada na maturidade para a melhoria dos processos de software.

² Nota da R.T.: o SEI faz parte da Universidade de Carnegie Mellon, nos EUA.

FIGURA 2.12 Níveis de maturidade da capacidade.



PONTOS-CHAVE

- ▶ Processos de software são as atividades envolvidas na produção de um sistema de software. Os modelos de processos de software são representações abstratas desses processos.
- ▶ Modelos de processo gerais descrevem a organização dos processos de software. Exemplos gerais incluem o modelo em cascata, o desenvolvimento incremental e a configuração e integração de componentes reusáveis.
- ▶ A engenharia de requisitos é o processo de desenvolver uma especificação de software. Especificações se destinam a comunicar as necessidades do cliente para os desenvolvedores do sistema.
- ▶ Os processos de projeto (*design*) e implementação se preocupam em transformar uma especificação de requisitos em um sistema de software executável.
- ▶ A validação de software é o processo de conferir se o sistema está em conformidade com a sua especificação e se isso satisfaz as necessidades reais dos usuários do sistema.
- ▶ A evolução do software ocorre quando você modifica os sistemas de software existentes para atender a novos requisitos. As mudanças são contínuas e o software deve evoluir para continuar sendo útil.
- ▶ Os processos devem incluir atividades para lidar com a mudança. Isso pode envolver uma fase de prototipação, que ajuda a evitar as más decisões de requisitos e projeto (*design*). Os processos podem ser estruturados para desenvolvimento e entrega iterativos, de modo que as mudanças possam ser feitas sem perturbar o sistema como um todo.
- ▶ A melhoria dos processos é a atividade de aperfeiçoar os processos de software existentes, elevando sua qualidade, diminuindo os custos ou reduzindo o tempo de desenvolvimento. Trata-se de um processo cíclico que envolve medição, análise e modificação de processos.

LEITURAS ADICIONAIS

"Process models in software engineering." É uma excelente visão geral dos modelos de processo em engenharia de software que foram propostos. SCACCHI, W. *Encyclopaedia of software engineering*. J. J. Marciniak, John Wiley & Sons, 2001. Disponível em: <<http://www.ics.uci.edu/~wscacchi/Papers/SE-Encyc/Process-Models-SE-Encyc.pdf>>. Acesso em: 30 mar. 2018.

Software process improvement: results and experience from the field. Esse livro é uma coleção de artigos concentrados em estudos de caso de melhoria de processos em várias empresas pequenas e médias da Noruega. Também inclui uma boa introdução às questões

gerais da melhoria de processos. CONRAD, R.; DYBÅ, T.; SJØBERG, D.; ULSUND, T. Springer, 2006.

"Software development life cycle models and methodologies." Essa postagem em *blog* é um resumo sucinto de vários modelos de processo de software que foram propostos e utilizados. A postagem discute as vantagens e desvantagens de cada um desses modelos. SAMI, M.; 2012. Disponível em: <<http://melsatar.wordpress.com/2012/03/15/software-development-life-cycle-models-and-methodologies/>>. Acesso em: 30 mar. 2018.

SITE³

Apresentações em PowerPoint para este capítulo disponíveis em: <<http://software-engineering-book.com/slides/chap2/>>. Links para vídeos de apoio disponíveis em: <<http://software-engineering-book.com/videos/software-engineering/>>.

³ Todo o conteúdo disponibilizado na seção *Site* de todos os capítulos está em inglês.

EXERCÍCIOS

- 2.1. Sugira o modelo de processo de software genérico mais apropriado que poderia ser utilizado como uma base para gerenciar o desenvolvimento dos seguintes sistemas. Justifique sua resposta de acordo com o tipo de sistema que está sendo desenvolvido:
 - ▶ Um sistema para controlar o freio ABS em um carro.
 - ▶ Um sistema de realidade virtual para apoiar a manutenção de software.
 - ▶ Um sistema contábil de uma universidade que substitua um sistema existente.
 - ▶ Um sistema interativo de planejamento de viagem que ajude os usuários a planejar suas viagens com o menor impacto ambiental.
- 2.2. Explique por que o desenvolvimento incremental é a abordagem mais eficaz para desenvolver sistemas de software de negócio. Por que esse modelo é menos apropriado para a engenharia de sistemas de tempo real?
- 2.3. Considere o modelo de processo de integração e configuração exibido na Figura 2.3. Explique por que é essencial repetir a atividade de engenharia de requisitos no processo.
- 2.4. Sugira por que é importante fazer uma distinção entre desenvolver os requisitos do usuário e desenvolver os requisitos do sistema no processo de engenharia de requisitos.
- 2.5. Usando um exemplo, explique por que as atividades de projeto de arquitetura, banco de dados, interface e componentes são interdependentes.
- 2.6. Explique por que o teste de software deve sempre ser uma atividade incremental, em etapas. Os programadores são as melhores pessoas para testar os programas que desenvolveram?
- 2.7. Explique por que a mudança é inevitável nos sistemas complexos e cite exemplos de atividades de processo de software (além da prototipação e entrega incremental) que podem ajudar a prever as possíveis mudanças e tornar o software que está sendo desenvolvido mais tolerante à mudança.
- 2.8. Você desenvolveu um protótipo de um sistema de software e sua gerente está muito impressionada com ele. Ela propõe que ele seja colocado em uso como um sistema em produção, com novas características adicionadas conforme a necessidade. Isso evita o gasto de desenvolvimento do sistema e torna-o imediatamente útil. Escreva um relatório curto para a sua gerente explicando por que os sistemas prototipados não devem ser utilizados como sistemas de produção.
- 2.9. Sugira duas vantagens e duas desvantagens da abordagem de avaliação e melhoria de processo incorporada ao modelo de maturidade da capacidade do SEI.
- 2.10. Historicamente, a introdução da tecnologia causou profundas mudanças no mercado de trabalho e, ao menos temporariamente, tirou as pessoas de seus empregos. Discuta se a automação extensiva de processos tende a ter as mesmas consequências para os engenheiros de software. Se você acha que não, explique o porquê. Se você acha que vai reduzir as oportunidades de trabalho, é ético que os engenheiros afetados resistam ativamente ou passivamente à introdução dessa tecnologia?

REFERÊNCIAS

- ABRIAL, J. R. *The B book: assigning programs to meanings*. Cambridge, UK: Cambridge University Press, 2005.
- _____. *Modeling in Event-B: system and software engineering*. Cambridge, UK: Cambridge University Press, 2010.
- BOEHM, B. W. "A Spiral Model of Software Development and Enhancement." *IEEE Computer*, v. 21, n. 5, 1988, p. 61-72. doi:10.1145/12944.12948.
- BOEHM, B. W.; TURNER, R. "Balancing agility and discipline: evaluating and integrating agile and plan-driven methods." In: *26ª Conferência Internacional de Engenharia de Software*. Edimburgo, Escócia, 2004. doi:10.1109/ICSE.2004.1317503.
- CHRISIS, M. B.; KONRAD, M.; SHRUM, S. *CMMI for development: guidelines for process integration and product improvement*. 3. ed. Boston: Addison-Wesley, 2011.
- HUMPHREY, W. S. "Characterizing the software process: a maturity framework." *IEEE Software*, v. 5, n. 2, 1988, p. 73-79. doi:10.1109/2.59.
- KOSKELA, L. *Effective unit testing: a guide for Java developers*. Greenwich, CT: Manning Publications, 2013.
- KRUTCHEN, P. *The rational unified process — An introduction*. 3. ed. Reading, MA: Addison-Wesley, 2003.
- ROYCE, W. W. "Managing the development of large software systems: concepts and techniques." In: *IEEE WESTCON*. Los Angeles, CA, 1970. p. 1-9.
- WHEELER, W.; J. WHITE. *Spring in practice*. Greenwich, CT: Manning Publications, 2013.